CI/CD Research and Automated Testing – Team Wack BucHunt

1. Continuous Integration and Continuous Deployment (CI/CD)

Our team researched implementing a CI/CD pipeline using GitHub Actions, GitHub Container Registry (GHCR), and Docker Compose to automate the testing, building, and deployment of BucHunt.

The proposed process is:

1. A developer merges code into the `sprint4` branch.
2. GitHub Actions automatically runs tests and builds the Docker images.
3. If the tests and builds succeed, GitHub Actions publishes the new images to GHCR.
4. A scheduled deployment script on our Linux server checks GHCR for updated images.
5. If newer images are available, the server downloads them.
6. Docker Compose recreates the affected containers using the new images.
7. The updated BucHunt application becomes available without requiring manual deployment.

The Linux server initiates outbound connections to retrieve the images, eliminating the need to expose SSH publicly.

Example Deployment Timeline

Time	Event
12:00	Developer merges code into `sprint4`
12:01	GitHub Actions builds the Docker image
12:02	GitHub Actions publishes the image to GHCR
12:03	Linux deployment script downloads the updated image
12:03	Docker Compose recreates the affected container
12:04	Updated BucHunt application is available

These times are illustrative. Actual deployment times depend on build duration, image size, and server availability.

Benefits

- Eliminates manual rebuilding and deployment.
- Automatically runs tests before publishing new images.
- Keeps Docker builds off the production server.
- Allows deployments without exposing SSH to the internet.
- Provides a repeatable deployment process and a record of build results through GitHub Actions.

2. Existing Automated Backend Tests

We also reviewed an existing automated integration test for the BucHunt backend. The test uses Node.js's built-in testing framework to verify that hunt tasks are correctly retrieved from CSV files through the backend API.

How the Test Works

1. Creates a temporary directory for test data.
2. Finds an available network port.
3. Starts a separate instance of the BucHunt backend (`index.js`) in test mode.
4. Waits for the backend to respond.
5. Creates sample hunt data and sends HTTP requests.
6. Compares the API responses against expected results.
7. Stops the test server and removes the temporary files.

Existing Test Cases

Test	Purpose	Expected Result
Missing Tasks	Request tasks for a hunt that does not exist	HTTP 404 with `"Hunt not found."`
Empty Tasks	Create a hunt containing zero tasks	HTTP 200 with an empty task array
Ten Tasks	Create a hunt containing 10 tasks	HTTP 200 with 10 tasks and unique sequential IDs
Hundred Tasks	Create a hunt containing 100 tasks	HTTP 200 with 100 tasks, unique sequential IDs, and no exposed answers

The fourth test also checks that the API does not return the correct task answers to players. This is important because players should not be able to discover answers by inspecting API responses.

Limitations of Existing Tests

The current tests do not verify:

- Correct or incorrect answer submissions.
- Multiple players joining the same hunt.
- Tracking completed tasks.
- Frontend functionality.
- Docker builds or deployment.
- Data persistence after a server restart.

Additionally, the tests create CSV files directly rather than creating hunts through the application's API. Therefore, they verify task retrieval but not the complete hunt creation process.

3. Integrating Tests into CI/CD

The existing backend tests could be incorporated into GitHub Actions as part of the CI/CD pipeline.

When new code is pushed, GitHub Actions would run the automated tests before building and publishing updated Docker images.

If the tests pass, the workflow continues with image creation and publication to GHCR. If any tests fail, the workflow stops and prevents the failing version from being published through that workflow.

This approach helps prevent changes that break existing functionality from reaching the deployed application.

4. Conclusion

Our research identified GitHub Actions, GHCR, and Docker Compose as a potential solution for automating BucHunt deployments.

We also reviewed existing backend integration tests that could support continuous integration by validating task retrieval and API responses.

Combining automated testing with containerized deployment would reduce manual work, improve deployment consistency, and help the team identify problems before new versions are released.

The proposed CI/CD pipeline remains a researched implementation approach rather than a confirmed completed deployment.